WEEK 6: PRODUCT REVIEW AND RATING MANAGEMENT SYSTEM

Introduction:

Customer feedback is an important part of an e-commerce application because it helps customers understand product quality before making a purchase. Reviews and ratings also help businesses identify customer satisfaction and improve products when necessary.

In this practical, a Product Review and Rating Management System was developed using Oracle SQL. A `REVIEW` table was created and connected to the existing `CUSTOMER` and `PRODUCT` tables. Customer ratings and comments were stored, and SQL queries were used to retrieve reviews, calculate average ratings, and analyze product performance.

Creation of the REVIEW Table:

```
SQL> CREATE TABLE REVIEW (
  2      REVIEW_ID VARCHAR2(7),
  3      CUSTOMER_ID NUMBER,
  4      PRODUCT_ID VARCHAR2(7),
  5      RATING NUMBER,
  6      REVIEW_COMMENT VARCHAR2(200),
  7      REVIEW_DATE DATE,
  8      CONSTRAINT PK_REVIEW PRIMARY KEY (REVIEW_ID),
  9      CONSTRAINT FK_REVIEW_CUSTOMER
 10          FOREIGN KEY (CUSTOMER_ID)
 11          REFERENCES CUSTOMER(CUSTOMER_ID),
 12      CONSTRAINT FK_REVIEW_PRODUCT
 13          FOREIGN KEY (PRODUCT_ID)
 14          REFERENCES PRODUCT(PRODUCT_ID));

Table created.
```

```
SQL> DESC REVIEW;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 REVIEW_ID                                 NOT NULL VARCHAR2(7)
 CUSTOMER_ID                                        NUMBER
 PRODUCT_ID                                         VARCHAR2(7)
 RATING                                             NUMBER
 REVIEW_COMMENT                                     VARCHAR2(200)
 REVIEW_DATE                                        DATE
```

```
SQL> ALTER TABLE REVIEW ADD CONSTRAINT CHK_REVIEW_RATING CHECK (RATING BETWEEN 1 AND 5);

Table altered.
```

The primary key uniquely identifies every review. The foreign keys establish connections between reviews, customers, and products. The CHECK constraint ensures that the rating remains within the permitted range of 1 to 5.

Inserting Customer Reviews:

After creating the table, 15 review records were inserted into the database.

Each record contains a review identifier, customer identifier, product identifier, rating, feedback comment, and review date.

The ratings range from 1 to 5, allowing the system to store both positive and negative customer feedback.

```
SQL> INSERT INTO REVIEW VALUES('&REVIEW_ID',&CUSTOMER_ID,'&PRODUCT_ID',&RATING,'&REVIEW_COMMENT',DATE '&REVIEW_DATE'
);
Enter value for review_id: R001
Enter value for customer_id: 1
Enter value for product_id: P001
Enter value for rating: 5
Enter value for review_comment: Excellent product and good quality
Enter value for review_date: 2026-09-01
old   1: INSERT INTO REVIEW VALUES('&REVIEW_ID',&CUSTOMER_ID,'&PRODUCT_ID',&RATING,'&REVIEW_COMMENT',DATE '&REVIEW_D
ATE')
new   1: INSERT INTO REVIEW VALUES('R001',1,'P001',5,'Excellent product and good quality',DATE '2026-09-01')

1 row created.
```

```
SQL> SELECT * FROM REVIEW;

REVIEW_ID  CUSTOMER_ID PRODUCT_ID   RATING REVIEW_COMMENT                     REVIEW_DATE
---------- ----------- ------------ ------ ---------------------------------- -----------
R001                1 P001              5 Excellent product and good quality  01-SEP-26
R002                2 P002              4 Good product for the price          02-SEP-26
R003                3 P003              5 Very useful and easy to use         03-SEP-26
R004                4 P004              3 Average product quality             04-SEP-26
R005                5 P005              4 Satisfied with the purchase         05-SEP-26
R006                6 P006              2 Product quality needs improvement   06-SEP-26
R007                7 P007              5 Excellent performance and quality   07-SEP-26
R008                8 P008              3 Product is acceptable               08-SEP-26
R009                9 P009              4 Good product and fast delivery      09-SEP-26
R010               10 P010              5 Amazing product experience          10-SEP-26
R011               11 P011              1 Very poor product quality           11-SEP-26
R012               12 P012              4 Worth buying                        12-SEP-26
R013               13 P013              5 Highly recommended product          13-SEP-26
R014               14 P014              2 Not satisfied with the product      14-SEP-26
R015               15 P015              4 Good quality and reasonable price   15-SEP-26

15 rows selected.
```

Updating and Deleting Reviews:

The database supports modifications to existing feedback and removal of reviews when necessary.

Updating a Review

A customer's feedback may need to be corrected or modified. The `UPDATE` statement changes the comment associated with a particular review.

```
SQL> UPDATE REVIEW
  2  SET REVIEW_COMMENT = 'Good quality and value for money'
  3  WHERE REVIEW_ID = 'R002';

1 row updated.
```

```
SQL> SELECT *
  2  FROM REVIEW
  3  WHERE REVIEW_ID = 'R002';

REVIEW_ID  CUSTOMER_ID PRODUCT_ID   RATING REVIEW_COMMENT                    REVIEW_DATE
---------- ----------- ------------ ------ --------------------------------- -----------
R002                2 P002              4 Good quality and value for money  02-SEP-26
```

Deleting a Review:

A review that is inappropriate or invalid may need to be removed from the database.

```
SQL> DELETE FROM REVIEW
  2  WHERE REVIEW_ID = 'R011';

1 row deleted.
```

```
SQL> SELECT *
  2  FROM REVIEW
  3  WHERE REVIEW_ID = 'R011';

no rows selected
```

Retrieving Product Review Details:

SQL queries were used to retrieve customer feedback based on different conditions.

1. Displaying Reviews for a Specific Product

The following query displays the reviews submitted for product `P001`.

```
SQL> SELECT *
  2  FROM REVIEW
  3  WHERE PRODUCT_ID = 'P001';

REVIEW_ID  CUSTOMER_ID PRODUCT_ID   RATING REVIEW_COMMENT                    REVIEW_DATE
---------- ----------- ------------ ------ --------------------------------- -----------
R001                1 P001              5 Excellent product and good quality  01-SEP-26
```

2. Displaying Customer Names with Reviews

The `CUSTOMER` and `REVIEW` tables were joined to display customer names along with their feedback.

```
SQL> SELECT C.CUSTOMER_NAME,
  2         R.PRODUCT_ID,
  3         R.RATING,
  4         R.REVIEW_COMMENT,
  5         R.REVIEW_DATE
  6  FROM CUSTOMER C
  7  JOIN REVIEW R
  8  ON C.CUSTOMER_ID = R.CUSTOMER_ID
  9  ORDER BY C.CUSTOMER_NAME;

CUSTOMER_NAME                                      PRODUCT_ID   RATING REVIEW_COMMENT                       REVIEW_DATE
-------------------------------------------------- ---------- ------ ----------------------------------- ------------
Ajay Kumar                                         P008              3 Product is acceptable               08-SEP-26
Akash Kumar                                        P013              5 Highly recommended product          13-SEP-26
Anjali Devi                                        P005              4 Satisfied with the purchase         05-SEP-26
Arun Kumar                                         P001              5 Excellent product and good quality  01-SEP-26
Gayathri R                                         P014              2 Not satisfied with the product      14-SEP-26
Ishitha R                                          P015              4 Good quality and reasonable price   15-SEP-26
Karthik M                                          P004              3 Average product quality             04-SEP-26
Manoj R                                            P009              4 Good product and fast delivery      09-SEP-26
Meeena K                                           P007              5 Excellent performance and quality   07-SEP-26
Nithya S                                           P010              5 Amazing product experience          10-SEP-26
Priya Sharma                                       P002              4 Good quality and value for money    02-SEP-26
Rahul Raj                                          P003              5 Very useful and easy to use         03-SEP-26
Suresh Babu                                        P012              4 Worth buying                        12-SEP-26
Vijay Kumar                                        P006              2 Product quality needs improvement   06-SEP-26

14 rows selected.
```

3. Finding Products with the Most Reviews

The following query calculates the number of reviews received by each product.

```
SQL> SELECT PRODUCT_ID,
  2         COUNT(*) AS TOTAL_REVIEWS
  3  FROM REVIEW
  4  GROUP BY PRODUCT_ID
  5  ORDER BY TOTAL_REVIEWS DESC;

PRODUCT_ID   TOTAL_REVIEWS
---------- -------------
P009                     1
P010                     1
P001                     1
P012                     1
P004                     1
P007                     1
P014                     1
P013                     1
P006                     1
P008                     1
P015                     1
P005                     1
P002                     1
P003                     1

14 rows selected.
```

4. Displaying Recent Customer Feedback

The following query sorts reviews by date, with the newest records displayed first.

```
SQL> SELECT REVIEW_ID,
  2         CUSTOMER_ID,
  3         PRODUCT_ID,
  4         RATING,
  5         REVIEW_COMMENT,
  6         REVIEW_DATE
  7  FROM REVIEW
  8  ORDER BY REVIEW_DATE DESC;

REVIEW_ID  CUSTOMER_ID PRODUCT_ID   RATING REVIEW_COMMENT                      REVIEW_DATE
---------- ----------- ------------ ------ ------------------------------------ ------------
R015               15 P015              4 Good quality and reasonable price   15-SEP-26
R014               14 P014              2 Not satisfied with the product      14-SEP-26
R013               13 P013              5 Highly recommended product          13-SEP-26
R012               12 P012              4 Worth buying                        12-SEP-26
R010               10 P010              5 Amazing product experience          10-SEP-26
R009                9 P009              4 Good product and fast delivery      09-SEP-26
R008                8 P008              3 Product is acceptable               08-SEP-26
R007                7 P007              5 Excellent performance and quality   07-SEP-26
R006                6 P006              2 Product quality needs improvement   06-SEP-26
R005                5 P005              4 Satisfied with the purchase         05-SEP-26
R004                4 P004              3 Average product quality             04-SEP-26
R003                3 P003              5 Very useful and easy to use         03-SEP-26
R002                2 P002              4 Good quality and value for money    02-SEP-26
R001                1 P001              5 Excellent product and good quality  01-SEP-26

14 rows selected.
```

5. Retrieving Reviews with Ratings Above 4

The following query selects reviews with ratings greater than 4.

```
SQL> SELECT *
  2  FROM REVIEW
  3  WHERE RATING > 4;

REVIEW_ID  CUSTOMER_ID PRODUCT_ID   RATING REVIEW_COMMENT                      REVIEW_DATE
---------- ----------- ------------ ------ ------------------------------------ ------------
R001                1 P001              5 Excellent product and good quality  01-SEP-26
R003                3 P003              5 Very useful and easy to use         03-SEP-26
R007                7 P007              5 Excellent performance and quality   07-SEP-26
R010               10 P010              5 Amazing product experience          10-SEP-26
R013               13 P013              5 Highly recommended product          13-SEP-26

SQL>
```

Calculation of Average Product Ratings

Aggregate functions were used to analyze ratings and compare product performance.

Average Rating for Each Product

```
SQL> SELECT PRODUCT_ID,
  2         ROUND(AVG(RATING), 2) AS AVERAGE_RATING
  3  FROM REVIEW
  4  GROUP BY PRODUCT_ID
  5  ORDER BY AVERAGE_RATING DESC;

PRODUCT_ID   AVERAGE_RATING
---------- --------------
P003                    5
P013                    5
P007                    5
P001                    5
P010                    5
P005                    4
P015                    4
P002                    4
P012                    4
P009                    4
P004                    3
P008                    3
P014                    2
P006                    2

14 rows selected.
```

The `AVG()` function calculates the average rating for each product, while `ROUND()` limits the result to two decimal places.

This helps compare products based on their average customer ratings.

Counting Reviews per Product

```
SQL> SELECT PRODUCT_ID,
  2         COUNT(*) AS TOTAL_REVIEWS
  3  FROM REVIEW
  4  GROUP BY PRODUCT_ID
  5  ORDER BY TOTAL_REVIEWS DESC;

PRODUCT_ID   TOTAL_REVIEWS
---------- -------------
P009                     1
P010                     1
P001                     1
P012                     1
P004                     1
P007                     1
P014                     1
P013                     1
P006                     1
P008                     1
P015                     1
P005                     1
P002                     1
P003                     1

14 rows selected.
```

The `COUNT()` function calculates the number of reviews associated with each product.

This information helps identify products that receive more customer attention.

Identifying the Highest-Rated Products

The following query identifies products with the highest average rating.

```
SQL> SELECT PRODUCT_ID,
  2         ROUND(AVG(RATING), 2) AS AVERAGE_RATING
  3  FROM REVIEW
  4  GROUP BY PRODUCT_ID
  5  HAVING AVG(RATING) = (
  6      SELECT MAX(AVG_RATING)
  7      FROM (
  8          SELECT AVG(RATING) AS AVG_RATING
  9          FROM REVIEW
 10          GROUP BY PRODUCT_ID
 11      )
 12  );

PRODUCT_ID   AVERAGE_RATING
---------- --------------
P010                    5
P001                    5
P007                    5
P003                    5
P013                    5
```

The query calculates the average rating for each product and compares those averages to find the highest value.

Identifying Products with an Average Rating Above 4

```
SQL> SELECT PRODUCT_ID,
  2         ROUND(AVG(RATING), 2) AS AVERAGE_RATING
  3  FROM REVIEW
  4  GROUP BY PRODUCT_ID
  5  HAVING AVG(RATING) > 4
  6  ORDER BY AVERAGE_RATING DESC;

PRODUCT_ID   AVERAGE_RATING
---------- --------------
P010                    5
P001                    5
P013                    5
P003                    5
P007                    5
```

Generation of Product Review Reports

Three reports were generated to summarize product ratings and customer feedback.

Report 1: Product Rating Analysis

To display product names, review counts, and average ratings in a single report.

```
SQL> SELECT P.PRODUCT_NAME,
  2         COUNT(R.REVIEW_ID) AS NUMBER_OF_REVIEWS,
  3         ROUND(AVG(R.RATING), 2) AS AVERAGE_RATING
  4  FROM PRODUCT P
  5  LEFT JOIN REVIEW R
  6  ON P.PRODUCT_ID = R.PRODUCT_ID
  7  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
  8  ORDER BY AVERAGE_RATING DESC NULLS LAST;

PRODUCT_NAME         NUMBER_OF_REVIEWS AVERAGE_RATING
-------------------- ----------------- --------------
Bluetooth Speaker                    1              5
Face wash                            1              5
Office Chair                         1              5
Python programming                   1              5
Smart phone                          1              5
Denim Jeans                          1              4
Yoga Mat                             1              4
Running Shoes                        1              4
Wireless Mouse                       1              4
Cooking oil                          1              4
Electric Kettle                      1              3
Cotton Shirt                         1              3
Study Table                          1              2
Data Structures                      1              2
Building Blocks                      0

15 rows selected.
```

Report 2: Customer Feedback Analysis
To identify products that receive more reviews, products with good ratings, and products that may need improvement.

1. Most-Reviewed Products

```
SQL> SELECT P.PRODUCT_NAME,
  2         COUNT(R.REVIEW_ID) AS TOTAL_REVIEWS
  3  FROM PRODUCT P
  4  JOIN REVIEW R
  5  ON P.PRODUCT_ID = R.PRODUCT_ID
  6  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
  7  ORDER BY TOTAL_REVIEWS DESC;

PRODUCT_NAME         TOTAL_REVIEWS
-------------------- -------------
Study Table                      1
Bluetooth Speaker                1
Cooking oil                      1
Smart phone                      1
Python programming               1
Running Shoes                    1
Yoga Mat                         1
Face wash                        1
Wireless Mouse                   1
Cotton Shirt                     1
Denim Jeans                      1
Data Structures                  1
Electric Kettle                  1
Office Chair                     1

14 rows selected.
```

2. Highly Rated Products

```
SQL> SELECT P.PRODUCT_NAME,
  2         ROUND(AVG(R.RATING), 2) AS AVERAGE_RATING
  3  FROM PRODUCT P
  4  JOIN REVIEW R
  5  ON P.PRODUCT_ID = R.PRODUCT_ID
  6  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
  7  HAVING AVG(R.RATING) >= 4
  8  ORDER BY AVERAGE_RATING DESC;

PRODUCT_NAME         AVERAGE_RATING
-------------------- --------------
Python programming                5
Smart phone                       5
Face wash                         5
Office Chair                      5
Bluetooth Speaker                 5
Denim Jeans                       4
Wireless Mouse                    4
Yoga Mat                          4
Cooking oil                       4
Running Shoes                     4

10 rows selected.
```

3. Products Requiring Improvement

```
SQL> SELECT P.PRODUCT_NAME,
  2         ROUND(AVG(R.RATING), 2) AS AVERAGE_RATING
  3  FROM PRODUCT P
  4  JOIN REVIEW R
  5  ON P.PRODUCT_ID = R.PRODUCT_ID
  6  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
  7  HAVING AVG(R.RATING) < 3
  8  ORDER BY AVERAGE_RATING;

PRODUCT_NAME         AVERAGE_RATING
-------------------- --------------
Data Structures                   2
Study Table                       2
```

Report 3: Rating Distribution

To examine how customer ratings are distributed across the products.

1. Number of Five-Star Ratings

```
SQL> SELECT COUNT(*) AS FIVE_STAR_RATINGS
  2  FROM REVIEW
  3  WHERE RATING = 5;

FIVE_STAR_RATINGS
-----------------
                5
```

2. Number of Four-Star Ratings

```
SQL> SELECT COUNT(*) AS FOUR_STAR_RATINGS
  2  FROM REVIEW
  3  WHERE RATING = 4;

FOUR_STAR_RATINGS
-----------------
                5
```

3. Identifying Low-Rated Products

```
SQL> SELECT P.PRODUCT_NAME,
  2         R.RATING,
  3         R.REVIEW_COMMENT
  4  FROM PRODUCT P
  5  JOIN REVIEW R
  6  ON P.PRODUCT_ID = R.PRODUCT_ID
  7  WHERE R.RATING <= 2
  8  ORDER BY R.RATING;

PRODUCT_NAME         RATING REVIEW_COMMENT
-------------------- ------ --------------------------------
Study Table               2 Not satisfied with the product
Data Structures           2 Product quality needs improvement
```

4. Complete Rating Distribution

```
SQL> SELECT RATING,
  2         COUNT(*) AS TOTAL_REVIEWS
  3  FROM REVIEW
  4  GROUP BY RATING
  5  ORDER BY RATING DESC;

RATING TOTAL_REVIEWS
------ -------------
     5             5
     4             5
     3             2
     2             2
```

Conclusion

In this practical, a **Product Review and Rating Management System** was implemented using Oracle SQL. The `REVIEW` table was created and connected to the existing customer and product tables through foreign key relationships.

Customer feedback was inserted, and SQL statements were used to update and delete reviews. Queries were also executed to retrieve product reviews, calculate average ratings, count reviews, and identify highly rated and low-rated products.

Finally, Product Rating Analysis, Customer Feedback Analysis, and Rating Distribution reports were generated.

Thus, the practical demonstrated how SQL can be used to manage customer feedback and analyze product performance in an e-commerce database.